

Cup Manipulator Tendon: Architecture, PD Control & ik-diagram Mode

Isaac-sim robotics notes

March 2026

1 File Map

File	Role
robots/cup_manipulator_tendon.py	CupManipulatorTendon, CupManipulatorIK, CupManipulatorPD
script_cup_manipulator_pendulam_with_spring_damper.py	IKDiagramSimulation — wires Drake diagram
test_drive_pulley.py	PulleyBase, CableRig (cable geometry)

2 CupManipulatorTendon Class

CupManipulatorTendon inherits from RobotBase and manages one `manipulator_cable.urdf` model (2 revolute DOF).

Constant	Value
JT1_NAME	"link1_base" (q_1 — direct drive)
JT2_NAME	"link2_link1" (q_2 — cable driven)
EE_OFFSET	[0.19, 0, 0.0515] m from link2_tendon
PULLEY_RADIUS	$r_p = 60 \times 5 \text{ mm} / (2\pi) \approx 47.75 \text{ mm}$

Setup sequence (before finalize):

```
manipulator.load_urdf_to_plant(plant, parser) # adds model instance
manipulator.weld_base_to_world(plant) # fixes base_mate to world frame
manipulator.add_joint_actuators(plant) # registers tau_link1_base, tau_link2_link1
manipulator.add_end_effector_frame(plant) # adds FixedOffsetFrame "tendon_ee"
plant.Finalize()
```

Key helper methods:

```
q = manipulator.get_positions_user_order(plant, ctx) # [q1, q2], rad
manipulator.set_positions_user_order(plant, ctx, [q1, q2])
p_ee = manipulator.get_end_effector_position(plant, ctx) # [x, y, z] world frame
```

3 Inverse Kinematics: CupManipulatorIK

`manipulator.ik` is a CupManipulatorIK instance.

3.1 Closed-form 2-R IK

Given target (t_x, t_y) and link lengths L_1, L_2 :

$$D = \frac{t_x^2 + t_y^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad |D| \leq 1 \quad (1)$$

$$q_2 = \arctan 2\left(\pm\sqrt{1-D^2}, D\right) \quad (2)$$

$$q_1 = \arctan 2(t_y, t_x) - \arctan 2(L_2 \sin q_2, L_1 + L_2 \cos q_2) \quad (3)$$

The sign $\pm$ selects elbow-up/down; the branch closest to `q_seed` is kept.

```
q_sol, ok = manipulator.ik.analytical(plant, target_xy=[0.4, 0.3],
                                     q_seed=np.zeros(2))
```

3.2 Geometric Jacobian (Drake + analytical)

$$\mathbf{J} = \frac{\partial \mathbf{p}_{ee}}{\partial \mathbf{q}} = \begin{bmatrix} -L_1 s_1 - L_2 s_{12} & -L_2 s_{12} \\ L_1 c_1 + L_2 c_{12} & L_2 c_{12} \end{bmatrix} \in \mathbb{R}^{2 \times 2} \quad (4)$$

where $s_i = \sin q_i$, $c_{12} = \cos(q_1 + q_2)$, etc. Verified against Drake's `CalcJacobianTranslationalVelocity` and finite-differences via `manipulator.ik.verify()`.

3.3 Hybrid Jacobian (cable actuation space)

Joint-2 actuation is the green-cable displacement $\ell_G = r_p q_2$. The mapping from actuation rates $\dot{\mathbf{u}} = [\dot{q}_1, \dot{\ell}_G]^\top$ to EE velocity:

$$\mathbf{A} = \text{diag}(1, r_p), \quad \mathbf{J}_h = \mathbf{J} \mathbf{A}^{-1} = \mathbf{J} \text{diag}\left(1, \frac{1}{r_p}\right) \quad (5)$$

```
J = manipulator.ik.jacobian(plant, ctx) # 2x2 geometric
J_h = manipulator.ik.jacobian_hybrid(plant, ctx) # 2x2 cable-space
q_dot = manipulator.ik.velocity(plant, ctx, ee_vel) # q-space vel IK
```

4 PD Controller: CupManipulatorIKSystem

`CupManipulatorIKSystem` is a Drake `LeafSystem`. It lives inside the Drake Diagram alongside the `MultibodyPlant`.

Input ports: `desired_ee_pos` [2], `plant_state` [n]

Output ports: `actuation` [2], `joint_positions` [2], `cable_lengths` [2], `cable_tensions` [2]

4.1 Joint 1 — Direct Drive

$$\tau_1 = K_{p1} (q_{1,\text{des}} - q_1) - K_{d1} \dot{q}_1 \quad [\text{Nm}] \quad (6)$$

```
tau1 = self._Kp1 * (q_des[0] - q[0]) - self._Kd1 * q_dot[0]
```

Default gains: $K_{p1} = 80 \text{ Nm/rad}$, $K_{d1} = 16 \text{ Nm}\cdot\text{s/rad} \Rightarrow \omega_n \approx 34 \text{ rad/s}$, overdamped.

4.2 Joint 2 — Single Cable with Tension Decomposition

A single cable wraps around the motor pulley (radius r_p). Both free ends attach to opposite sides of `link2`: one retracts, one extends. Cable displacement error and speed:

$$\Delta\ell = r_p (q_{2,\text{des}} - q_2), \quad \dot{\ell} = r_p \dot{q}_2 \quad (7)$$

Signed net cable force:

$$F_{\text{net}} = K_{p,c} \Delta\ell - K_{d,c} \dot{\ell} \quad [\text{N}] \quad (8)$$

Tension decomposition (cables can only pull, $T \geq 0$):

$$T_{\text{green}} = \max(F_{\text{net}}, 0), \quad T_{\text{red}} = \max(-F_{\text{net}}, 0) \quad (9)$$

Net joint torque via cable Jacobian $J_c = r_p$:

$$\tau_2 = (T_{\text{green}} - T_{\text{red}}) r_p = F_{\text{net}} r_p \quad [\text{Nm}] \quad (10)$$

```
delta_l = self._r_p * (q_des[1] - q[1]) # cable displacement error [m]
l_dot = self._r_p * q_dot[1] # cable speed [m/s]
F_net = self._Kp_cable * delta_l - self._Kd_cable * l_dot

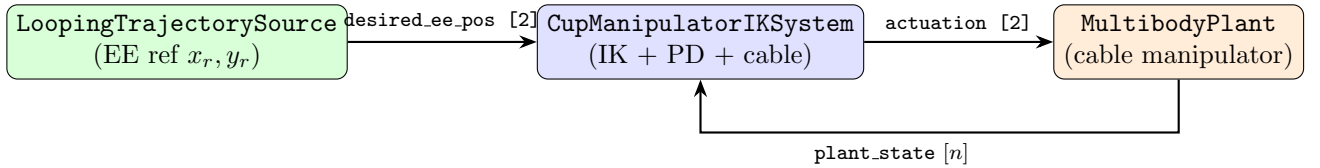
T_green = max(F_net, 0.0) # retracting side
T_red = max(-F_net, 0.0) # extending side

tau2 = (T_green - T_red) * self._r_p # = F_net * r_p [Nm]
tau = np.clip([tau1, tau2], -tau_max, tau_max)
```

Default cable gains: $K_{p,c} = 260 \text{ N/m}$, $K_{d,c} = 35 \text{ N}\cdot\text{s/m}$, $\tau_{\text{max}} = 10 \text{ Nm}$. Effective joint-space equivalents: $K_{p,j2} = K_{p,c} r_p^2$, $K_{d,j2} = K_{d,c} r_p^2$.

5 ik-diagram Mode (IKDiagramSimulation)

5.1 Drake Diagram Topology



All three blocks are added to Drake's `DiagramBuilder` and wired with `builder.Connect()`. The diagram is fixed-timestep ($\Delta t = 2 \text{ ms}$) with Meshcat streaming.

5.2 Build Sequence (in code)

```
sim = IKDiagramSimulation(CABLE_MANIPULATOR_CONFIG, meshcat)
sim.build_plant() # MultibodyPlant: URDF load -> weld -> actuators -> finalize
sim.build_trajectory(args) # PiecewisePolynomial circle/rect/line, clamped to workspace
sim.build_controller() # CupManipulatorIKSystem added to builder
sim.connect_and_build() # Connect(), add LogVectorOutput loggers, builder.Build()
sim.initialize() # patch zero-mass bodies, set q=0, verify Jacobian
sim.run() # AdvanceTo() in 0.1 s chunks, loops until Ctrl-C
sim.plot(args.traj_shape) # 4-panel time-series + EE XY overlay
```

5.3 Trajectory Shapes

The looping EE reference repeats every `--duration` seconds. Waypoints are clamped to the reachable workspace $[|L_1 - L_2|, L_1 + L_2]$.

<code>--traj-shape</code>	Formula	Key args
<code>circle</code>	$x = c_x + R \cos \theta, y = c_y + R \sin \theta$	<code>--traj-radius, --traj-cx, --traj-cy</code>
<code>rect</code>	Perimeter of $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$	<code>--traj-x-range, --traj-y-range</code>
<code>line</code>	$x \in [c_x - R, c_x + R], y = c_y$	same as circle

5.4 Loggers

Logger key	Signal
<code>state</code>	Full plant state $[\mathbf{q}, \dot{\mathbf{q}}]$
<code>q_des</code>	IK solution $[q_{1,\text{des}}, q_{2,\text{des}}]$
<code>cable</code>	Green/red cable displacements $[\Delta \ell_G, \Delta \ell_R]$ [m]
<code>tensions</code>	$[T_{\text{green}}, T_{\text{red}}]$ [N]
<code>ref</code>	EE reference (x_r, y_r)

5.5 Invocation Example

```
python script_cup_manipulator_pendulam_with_spring_damper.py \
  --mode ik-diagram \
  --traj-shape circle --traj-radius 0.30 --traj-cx 0.35 --traj-cy 0.0 \
  --duration 8.0 --traj-n 200
```